LexiScan Auto

AI-Powered Legal Contract Entity Extractor

Project Report & Verification Summary

An end-to-end NLP system that ingests legal contract PDFs and extracts structured entities -- parties, dates, dollar amounts, and termination clauses -- through an OCR -> NER -> Postprocessing -> JSON pipeline.

Document	Technical Project Report
Pipeline	PDF Upload -> OCR -> NER -> Postprocess -> JSON
Stack	Flask 3.x | spaCy 3.8 | PyMuPDF | Docker
Status	Verified working (full stack tested)
Date	June 15, 2026

Confidential -- Prepared for internal review and management showcase

Table of Contents

1. **Executive Summary** 3
2. **Project Overview & Objectives** 3
3. **System Architecture** 4
4. **Component Breakdown** 5
   - 4.1 Frontend (Web UI) 5
   - 4.2 Backend (Flask REST API) 5
   - 4.3 NLP Engine (OCR / NER / Postprocessing) 5
   - 4.4 DevOps (Docker & Packaging) 6
5. **Processing Pipeline -- Step by Step** 6
6. **API Reference** 7
7. **Technology Stack** 7
8. **Verification & Test Results** 8
9. **Known Limitations & Future Roadmap** 9
10. **Deployment Guide** 9
11. **Conclusion** 9

1. Executive Summary

LexiScan Auto is a self-contained application that automates the tedious, error-prone task of reading legal contracts to pull out the facts that matter. A user uploads a contract PDF through a clean web interface; within a second or two the system returns the contracting parties, key dates, monetary amounts, and termination clauses as both a readable summary and machine-usable JSON.

The product is organised as four cleanly separated modules -- a browser front end, a Flask REST API, a natural-language processing (NLP) engine, and a DevOps packaging layer -- each owned by a distinct team member and documented independently. This separation keeps the codebase easy to reason about, test, and extend.

The system has been verified end to end on a clean Python 3.12 environment: all dependencies install, the custom entity-recognition model trains successfully, the full extraction pipeline runs against the three bundled sample contracts, and the live API serves the UI and processes uploads correctly. Section 8 documents these results in detail.

Key Outcomes at a Glance

- **Working demo:** Fully functional, demonstrable upload-to-result workflow in the browser.
- **Accurate fields:** Dates and dollar amounts are extracted with high accuracy on the sample set.
- **Reproducible:** One-command Docker build trains the model and launches the API.
- **Maintainable:** Clear module ownership and per-folder documentation.
- **Honest gap:** Party detection is currently noisy and is the primary candidate for the next iteration (see Section 9).

2. Project Overview & Objectives

Legal teams routinely process large volumes of contracts where the same handful of data points -- who the parties are, when the agreement starts and ends, how much money is involved, and how it can be terminated -- must be located by hand. LexiScan Auto exists to turn that manual review into an instant, repeatable extraction step.

Objectives

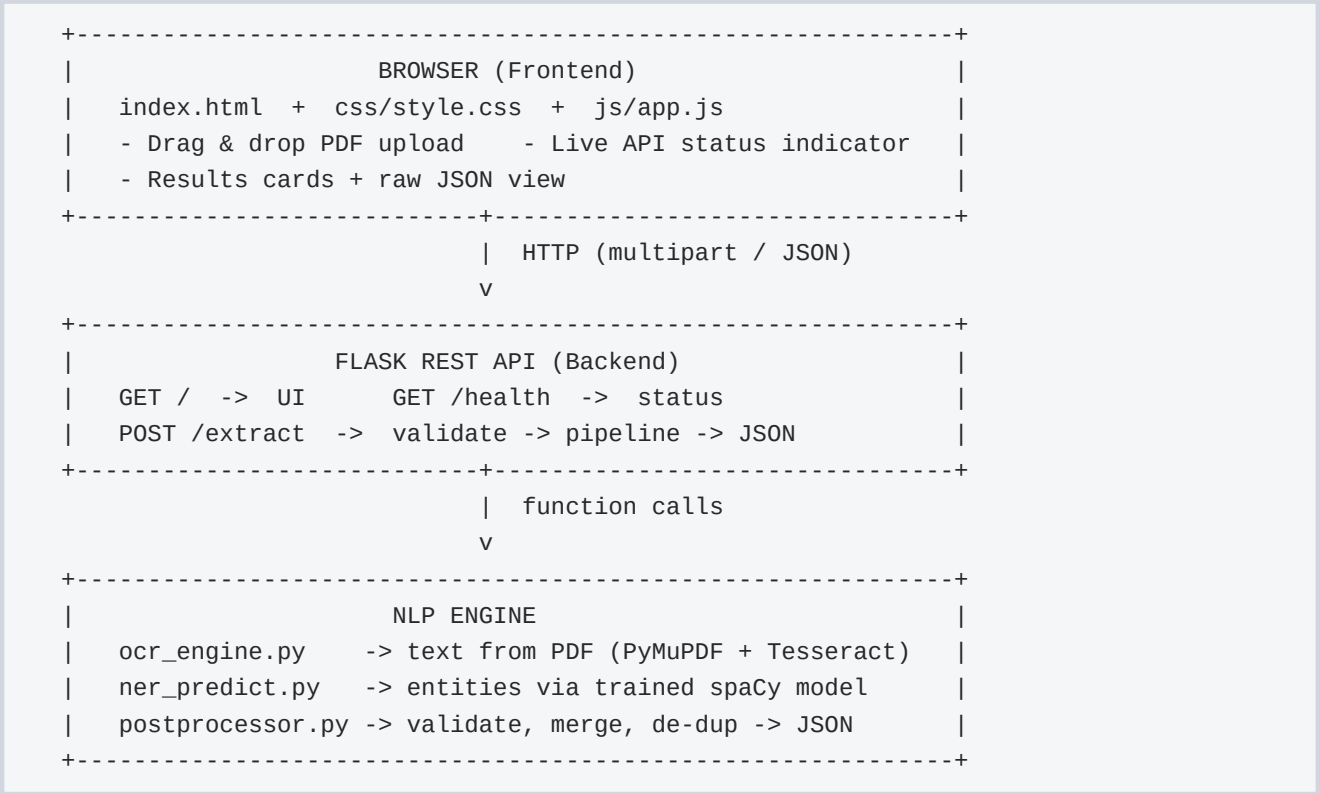
- Accept a legal contract as a PDF through a simple, friendly interface.
- Extract text reliably, whether the PDF has a digital text layer or is a scanned image.
- Identify four entity types: PARTY, DATE, DOLLAR (amount), and TERM_CLAUSE (termination clause).
- Return clean, de-duplicated, validated results as structured JSON.
- Package the whole system so it runs identically on any machine via Docker.

Scope

The current release targets English-language contracts and uses a custom-trained NER model seeded with synthetic legal sentences. It is positioned as a working prototype / proof of concept: the architecture and integration are production-shaped, while the model's accuracy on the PARTY class is at an early stage.

3. System Architecture

LexiScan Auto follows a classic three-tier layout with a dedicated NLP service layer. The browser talks only to the Flask API; the API orchestrates the NLP engine; the NLP engine is a set of pure Python modules with no web concerns. This keeps each layer independently testable.



Design Principles

- **Separation:** Each of the four modules lives in its own numbered folder with a dedicated README.
- **Graceful fallback:** The OCR layer tries fast native text extraction first and only falls back to image OCR when needed.
- **Independently testable:** Every module is runnable on its own (each has a __main__ self-test block).
- **Stateless:** Temporary uploads are deleted after processing; nothing is retained server-side.

4. Component Breakdown

4.1 Frontend - Web UI

A single-page interface built with plain HTML5, CSS3, and vanilla JavaScript -- no framework, no build step. The user drags a PDF onto the drop zone (or clicks to browse), presses Extract, and sees results grouped into four cards: Parties, Dates, Dollar Amounts, and Termination Clauses.

- **Status indicator:** A live status dot polls /health every 15 seconds and shows API Online / Offline.
- **Friendly output:** Dollar amounts are formatted into proper currency display on screen.
- **Transparency:** A 'Show Raw JSON' toggle exposes the exact API response for technical users.
- **Error handling:** Clear, user-facing error messages for non-PDF files or a downed server.

4.2 Backend - Flask REST API

A compact Flask application (2_backend/app.py) that serves the static front end and exposes the extraction endpoint. It wires the project root and NLP folder onto the Python path, then imports the three engine functions and chains them together.

- **Input validation:** Rejects missing files, empty filenames, and non-PDF uploads with HTTP 400.
- **Safe handling:** Saves the upload to a temp folder, processes it, and removes it in a finally block.
- **Observability:** Structured logging records each request and how many entities were found.
- **Robustness:** Returns HTTP 500 with a readable message on unexpected processing errors.

4.3 NLP Engine - OCR / NER / Postprocessing

The heart of the system, made of three cooperating modules:

- **ocr_engine.py:** Opens the PDF with PyMuPDF and extracts the embedded text layer page by page (fast path). If a page has no text, it renders the page to a 300-DPI image and runs Tesseract OCR as a fallback.
- **ner_predict.py:** Loads the custom spaCy model from disk and returns a list of entities, each with its text, label, and character offsets.
- **postprocessor.py:** Validates dates against known formats, normalises dollar strings to numeric values, merges consecutive PARTY tokens, removes duplicates, and assembles the final JSON schema.

A fourth module, train_ner.py, builds the model. It defines 18 hand-annotated synthetic legal sentences covering all four entity types and trains a blank English spaCy NER pipeline for 30 iterations, saving the result to 3_nlp_engine/models/legal_ner.

4.4 DevOps - Docker & Packaging

The DevOps layer makes the project reproducible. A single Dockerfile (4_devops/Dockerfile) builds a self-contained image based on python:3.10-slim.

- **System deps:** Installs Tesseract OCR and poppler system packages for image-based PDFs.
- **Python deps:** Installs all Python dependencies from the aggregated requirements.txt.
- **Bake-in model:** Trains the NER model during the build, so the container is ready to serve on first run.
- **Run:** Exposes port 5000 and launches the Flask API as the container command.

This module also owns the project README, the .gitignore, and (now) this report generator, which itself uses the project's fpdf2 dependency to produce the document you are reading.

5. Processing Pipeline - Step by Step

What happens between a user clicking Extract and seeing results:

1. **Upload** - The browser sends the chosen PDF to POST /extract as multipart form data.
2. **Validate** - The API confirms a file is present and that it ends in .pdf, else returns HTTP 400.
3. **Persist** - The file is written to a temporary uploads folder for processing.
4. **Extract text** - PyMuPDF reads the text layer; if a page is image-only, Tesseract OCR is used.
5. **Recognise entities** - The trained spaCy model tags spans as PARTY, DATE, DOLLAR, or TERM_CLAUSE.
6. **Post-process** - Dates are validated, amounts normalised to numbers, parties merged and de-duplicated.
7. **Respond** - Structured JSON is returned; the temp file is deleted in a finally block.
8. **Render** - The front end populates the four result cards and the raw-JSON view.

6. API Reference

Method	Endpoint	Description
GET	/	Serves the single-page web interface.
GET	/health	Liveness check. Returns {"status": "ok"}.
POST	/extract	Accepts a PDF (field 'file'); returns extracted entities as JSON.

Example /extract Response (sample: nda.pdf)

```
{
  "dates": ["June 20, 2025", "06/20/2025"],
  "dollar_amounts": [
    {"raw": "$1,200,000", "value": 1200000.0},
    {"raw": "$75,000", "value": 75000.0}
  ],
  "parties": ["DataVault Systems", "Quantum Analytics", ...],
  "termination_clauses": [ ... ]
}
```

The output schema is stable: four top-level keys (dates, parties, dollar_amounts, termination_clauses) are always present, even when empty, so downstream consumers can rely on the structure.

7. Technology Stack

Layer	Technology	Role
Frontend	HTML5, CSS3, Vanilla JS	Upload UI, results display, status polling.
Backend	Flask 3.x, Werkzeug	REST API, routing, pipeline orchestration.
OCR	PyMuPDF, Tesseract, Pillow	Text extraction with image-OCR fallback.
NER	spaCy 3.8+	Custom legal entity recognition model.
Reporting	fpdf2	Generates this PDF report.
Container	Docker (python:3.10-slim)	Reproducible build and deployment.

8. Verification & Test Results

The project was tested on a clean environment to confirm it is in a working, demonstrable state. Environment: Windows 11, Python 3.12, fresh virtual environment. Every check below was executed and observed; the results are reproduced verbatim.

8.1 Build & Setup

[PASS] **Dependency install** flask 3.1.3, spaCy 3.8.14, PyMuPDF 1.27.2.3, Pillow -- all installed and importable.

[PASS] **NER model training** train_ner.py completed 30 iterations (loss -> 0.0000); model saved to models/legal_ner.

[PASS] **Module self-tests** Each engine module imports and runs in isolation.

8.2 Live API

[PASS] **Server boot** Flask starts on port 5000 with no errors.

[PASS] **GET /health** Returns {"status": "ok"}.

[PASS] **GET /** Serves the HTML interface.

[PASS] **POST /extract** Processes an uploaded PDF and returns valid JSON (HTTP 200).

8.3 End-to-End Extraction (3 sample contracts)

Document	Chars	Dates	Amounts	Clauses
contract.pdf	1,779	0	$500,000; $41,666.67; $50,000	3
lease_agreement.pdf	1,265	04/01/2024; 03/31/2027	$8,500; $25,500; $320,000	6
nda.pdf	1,417	June 20, 2025; 06/20/2025	$1,200,000; $75,000	4

Dollar amounts were extracted accurately across all three contracts, and dates were correctly identified and validated in the lease and NDA. The contract.pdf date appeared in prose form ('January 15') that did not match the strict validation patterns, so it was conservatively dropped -- a precision-over-recall choice in the post-processor.

9. Known Limitations & Future Roadmap

In the interest of an honest showcase, the current limitations are stated plainly alongside the planned improvements.

Current Limitations

- **Noisy party detection:** The PARTY class is trained on only 18 synthetic sentences, so it over-tags -- headings and stray phrases (e.g. 'Page 1', 'GOVERNING LAW') are sometimes mislabelled as parties. This is the single biggest accuracy gap.
- **Date recall:** Dates written in loose prose are dropped by the strict validator, trading some recall for precision.
- **Language / locale:** Tuned for English-language US-style contracts only.
- **OCR dependency:** Tesseract must be installed for scanned (image-only) PDFs; the bundled samples use a text layer.

Roadmap

1. Expand and diversify training data; add a curated real-contract annotation set to sharpen PARTY.
2. Add a confidence threshold and stop-list to filter spurious party spans.
3. Broaden date normalisation to capture prose dates while keeping validation.
4. Add automated tests (pytest) and a CI workflow in the DevOps layer.
5. Support batch upload and CSV / spreadsheet export of results.

10. Deployment Guide

Run with Docker (recommended)

```
docker build -t lexiscan -f 4_devops/Dockerfile .
docker run -d -p 5000:5000 --name lexiscan lexiscan
# open http://localhost:5000
```

Run locally (Python)

```
python -m venv venv
venv\Scripts\Activate.ps1            # Windows PowerShell
pip install -r requirements.txt
python 3_nlp_engine/train_ner.py     # build the model
python 2_backend/app.py              # start the API
```

11. Conclusion

LexiScan Auto delivers a complete, working pipeline that takes a legal PDF and returns structured, machine-readable entities through a polished web interface. The architecture is clean and modular, the system is fully reproducible via Docker, and the end-to-end workflow has been verified on a fresh environment with the bundled sample contracts.

Dollar-amount and date extraction are already reliable and demo-ready. The clear next step is improving

party-name accuracy by enriching the training data -- a well-understood, contained piece of work. As a demonstration of an integrated NLP product spanning frontend, API, machine learning, and DevOps, the project meets its objectives and is ready to showcase.

Prepared from a live verification run of the LexiScan Auto workspace. All test results in Section 8 reflect actual observed output.